TECHNICAL REPORT

Aegis

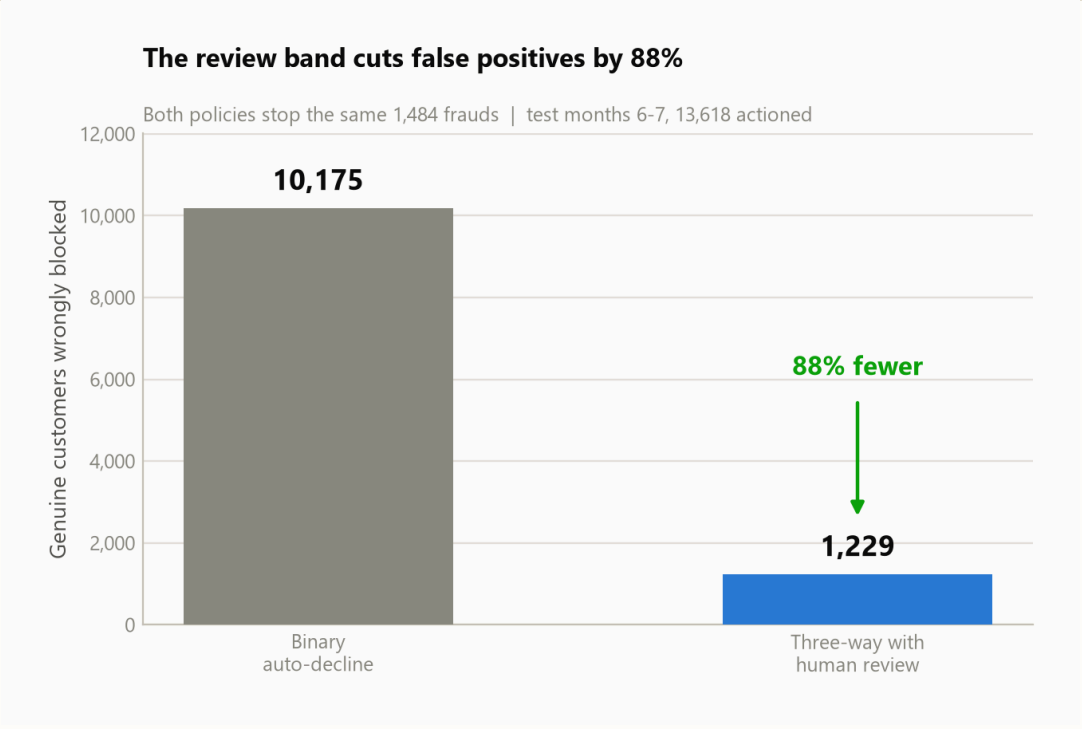
Adaptive application-fraud decisioning for digital lending

Fraud detection is a decision problem, not a classification problem. This report argues that the interesting question is not how well a model ranks applications, but where the decision boundaries come from — and shows what changes when they are derived from cost rather than tuned.

−88%
genuine customers wrongly blocked, at identical fraud detection

98%
reduction in calibration error, without which cost-derived thresholds are meaningless

41%
of model gain from behavioural signals rather than application form fields



Dataset	Bank Account Fraud Suite (NeurIPS 2022), Feedzai — 1,000,000 applications across 8 months
Model	LightGBM, 264 boosting iterations, 35 features, isotonic calibration
Evaluation	Temporal hold-out — months 6–7, never seen in training or calibration
Stack	FastAPI · React 19 · PostgreSQL 17 + pgvector · SHAP · Gemini
Source	`github.com/Fearless2389/Fraud-Detection-for-online-Lending`

1 The problem, and why the obvious framing is wrong

Digital lending removed the branch visit. It also removed every informal check that came with it — and origination fraud is the result.

An application arrives through a phone in under two minutes. There is no cashier reading a face, no manager recognising a name. The lender has a form, a device, a session, and a few seconds to decide. Fraud at this moment is not a stolen card being swiped; it is a **fabricated or hijacked identity asking to be given credit**, and once the money leaves it does not come back.

The obvious framing is: build a classifier, find fraud. That framing produces systems that fail in production, for a reason that is arithmetic rather than technical.

The asymmetry that governs everything

Fraud in this dataset runs at 1.40% of applications. So for every fraudulent application there are roughly seventy genuine ones. A model that flags 5% of traffic to catch half the fraud is *also* flagging thousands of real customers — and each of those is a person who wanted a loan, was treated as a criminal, and will not come back.

> THE COST STRUCTURE
>
> Blocking a genuine customer and approving a fraudster are **not symmetric errors**, and neither are they comparable in magnitude. A wrongful block costs the lender an acquisition and a reputation. An approved fraud costs the principal. In this system those are configured as `₹1,500` and `₹45,000` — a thirty-fold difference.

Once the errors are that asymmetric, the threshold matters more than the model. A binary system has one dial, and every position on it trades real customers against real losses. Most prototypes leave that dial at 0.5, which on this data misses 97% of all fraud — a number that sounds like a modelling failure but is really a threshold that was never chosen at all.

What this system does instead

Aegis makes three commitments, and the rest of this report is the evidence for each:

1. **The boundaries are derived, not tuned.** Two thresholds fall out of four business costs by minimising expected loss. Changing the lender's risk appetite is arithmetic, not a retrain.
2. **There are three outcomes, not two.** Approve, review, block. The review band is where the false-positive reduction comes from, and it is capped by how many analysts actually exist.
3. **Every decision is explained and recorded.** Additive SHAP attributions become adverse-action reason codes; each decision is written to an audit log the database will not let anyone rewrite.

A note on what is *not* claimed. This system addresses fraud at origination — identity fabrication and takeover. It does not address credit risk (a real applicant who cannot repay) or first-party bust-out fraud, which manifests months after approval and leaves no trace in an application form.

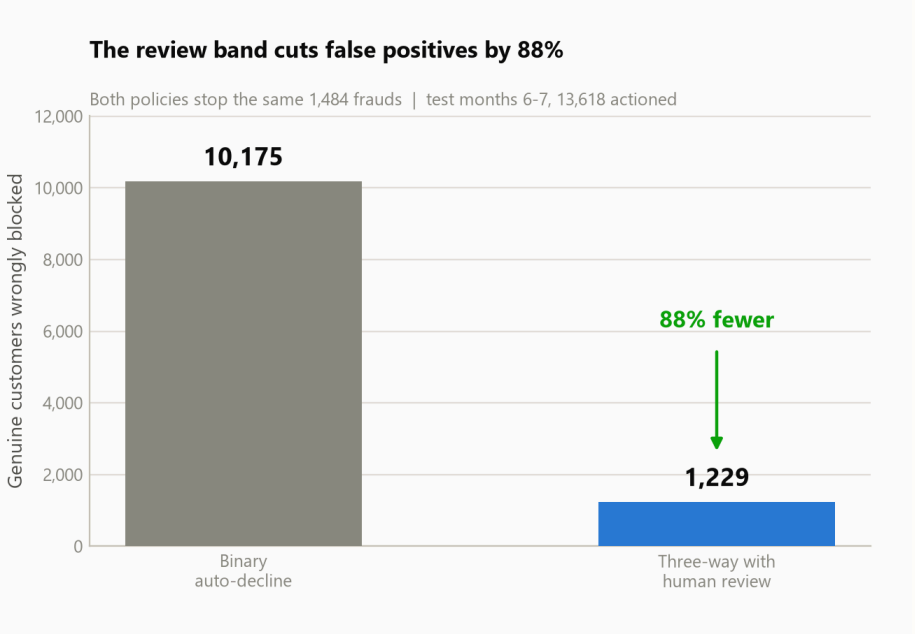
2 The headline result

At identical fraud detection, the three-way policy blocks 88% fewer genuine customers.

The comparison below is deliberately constructed to be hard on this system. The binary baseline is not a strawman left at 0.5 — it is **tuned to catch exactly as much fraud** as the three-way policy does. Both stop 1,484 frauds. The only question is what each destroys on the way.

POLICY ON HELD-OUT MONTHS 6–7	FRAUDS STOPPED	GENUINE BLOCKED	TO REVIEW
Binary auto-decline, tuned to equal recall	1,484	10,175	–
Three-way with a cost-derived review band	1,484	1,229	11,869
Genuine customers spared	no change	−8,946	

Equal-recall comparison. Measured on 205,011 held-out applications containing 2,878 frauds.



The mechanism is not a better model — it is the same model given a third option. Borderline applications route to an analyst who can clear a good customer, instead of being auto-declined by a cut-off that cannot tell "suspicious" from "guilty".

> THE HONEST CAVEAT
>
> This buys 8,946 customers at the price of 11,869 manual reviews — about 5.8% of traffic. That is only a good trade if the analysts exist. Section 7 shows what happens to the queue when they do not, and it is the weakest point in the design.

3 Finding I — the thresholds are arithmetic, not folklore

Ask most fraud prototypes where their threshold came from and the answer is 0.5, or "whatever maximised F1". Neither has a defensible justification, and neither survives the question a risk owner will actually ask: *why is the line there and not somewhere else?*

Aegis has an answer. Given the cost of a false positive (C_{fp}), a false negative (C_{fn}), a manual review (C_{rev}) and the rate at which analysts correctly identify fraud once they see it (r), the expected cost of each action is a line in the fraud probability p. The optimal action is whichever line is lowest, and the boundaries are where they cross:

$$\tau_{review} = C_{rev} / (r \cdot C_{fn}) \qquad \tau_{block} = (C_{fp} - C_{rev}) / (C_{fn} \cdot (1 - r) + C_{fp})$$

With the costs configured here that yields $\tau_{review} = 0.0049$ and $\tau_{block} = 0.2167$. Nothing was searched. No validation set was consulted. Change a cost and the operating point moves immediately — the model is untouched, which is exactly the property a risk function needs and a retrained model cannot offer.

Bayes-optimal is not the same as operable

The unconstrained review threshold above routes an enormous share of traffic to human review, because the arithmetic assumes review capacity is unlimited. It is not. A second constraint caps the review band at the share of volume the team can actually work — here 5% — which raises the review threshold to `0.0449`.

> WHY THIS MATTERS MORE THAN IT SOUNDS
>
> A policy that is optimal on paper and unstaffable in practice is not optimal. The capacity cap is the difference between a threshold derived in a notebook and one that can be deployed on Monday — and it is applied *after* the economics, so the trade being made stays visible rather than being hidden inside a tuned constant.

Cost outcome on held-out data

OUTCOME, MONTHS 6–7	COST-DERIVED POLICY	UNTUNED 0.5 CUT-OFF
Frauds missed	1,287	2,795
Genuine customers blocked	1,229	68
False positive rate	0.61%	0.03%
Total expected cost	**₹6.70 cr**	**₹12.59 cr**

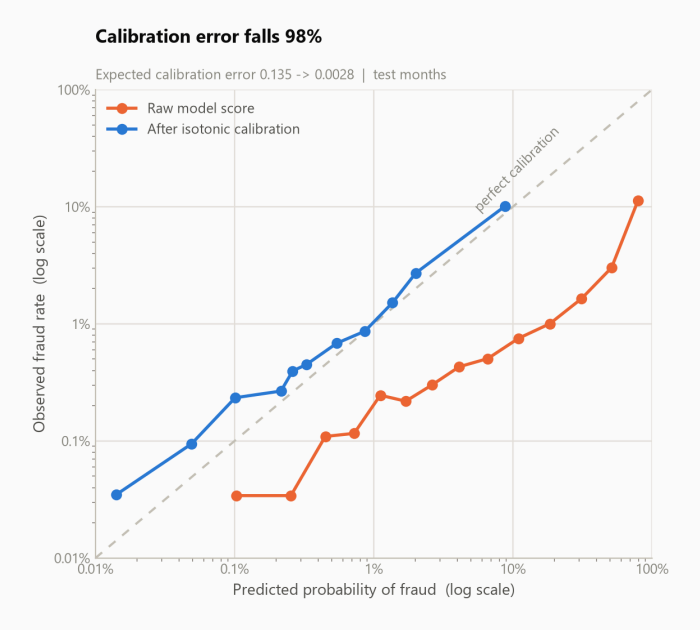
The 0.5 cut-off blocks almost nobody and therefore looks precise — while missing 97% of all fraud. It is cheaper on false positives and costs 1.9× as much overall.

4 Finding II — calibration is a precondition, not a polish step

Section 3 derives thresholds by comparing expected costs. That arithmetic multiplies a cost by a probability — which is only valid if the number the model emits *is* a probability. A raw gradient-boosting score is not. It ranks well and is systematically wrong in magnitude.

HELD-OUT MONTHS 6–7	RAW SCORE	ISOTONIC-CALIBRATED	CHANGE
Expected calibration error	0.1351	0.0028	-97.9%
Brier score	0.0724	0.0126	-82.6%
ROC-AUC (ranking — should not move)	0.8807	0.8803	-0.0004
Recall at a 5% alert budget	48.6%	55.3%	+6.6pp

Calibration barely moves ROC-AUC, and that is the point: it does not change the ordering of applications, only what the score *means*. Ranking metrics are blind to the defect it fixes.



Reliability on log-log axes, because almost all applications sit below 1% and a linear plot would compress every meaningful point into one corner. Before calibration the model is confidently wrong at the low end — exactly the region where the review threshold sits.

VERIFIED BY DELIBERATE FAILURE

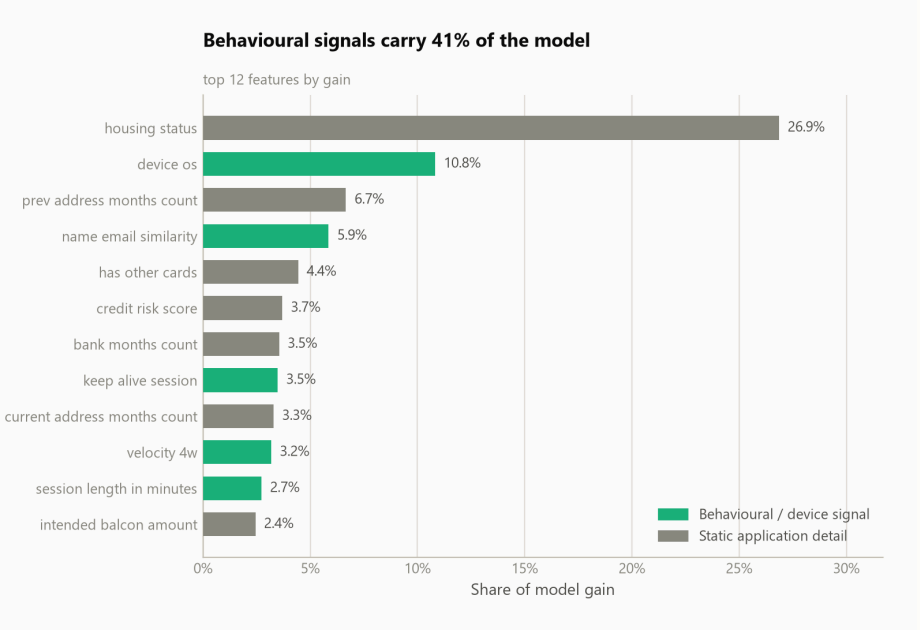
A regression test in this project feeds *uncalibrated* scores to the cost policy and asserts that it loses to a naive 0.5 baseline. It does. The test is kept permanently, because the failure is the argument: without calibration the entire decision layer of this system is invalid, and that should be provable rather than asserted.

5 Finding III — behavioural signal carries 41% of the model

A fraud model that reads only the application form is reading what the fraudster chose to write. The signals that are hard to fabricate are the ones the applicant did not know were being recorded: how the session behaved, how many distinct emails that device has seen, how many applications came from that postcode this month.

FEATURE	TYPE	SHARE OF GAIN
housing_status	application	26.9%
device_os	behavioural	10.8%
prev_address_months_count	application	6.7%
name_email_similarity	behavioural	5.9%
has_other_cards	application	4.4%
credit_risk_score	application	3.7%
bank_months_count	application	3.5%
keep_alive_session	behavioural	3.5%

Top eight features by total split gain, out of 35. The behavioural ones visible here account for 20.2% of total gain; the other 21.1% sits in the tail — velocity windows, postcode counts, device-email counts — which is why the model's behavioural share is 41.3% and not the sum of this column. Classification follows the model's own feature groups, not a list restated for this document.



Housing status dominates, which is worth stating plainly rather than hiding: it is a proxy for stability, and a model leaning this hard on one categorical field is a model whose fairness properties need checking — see Section 11.

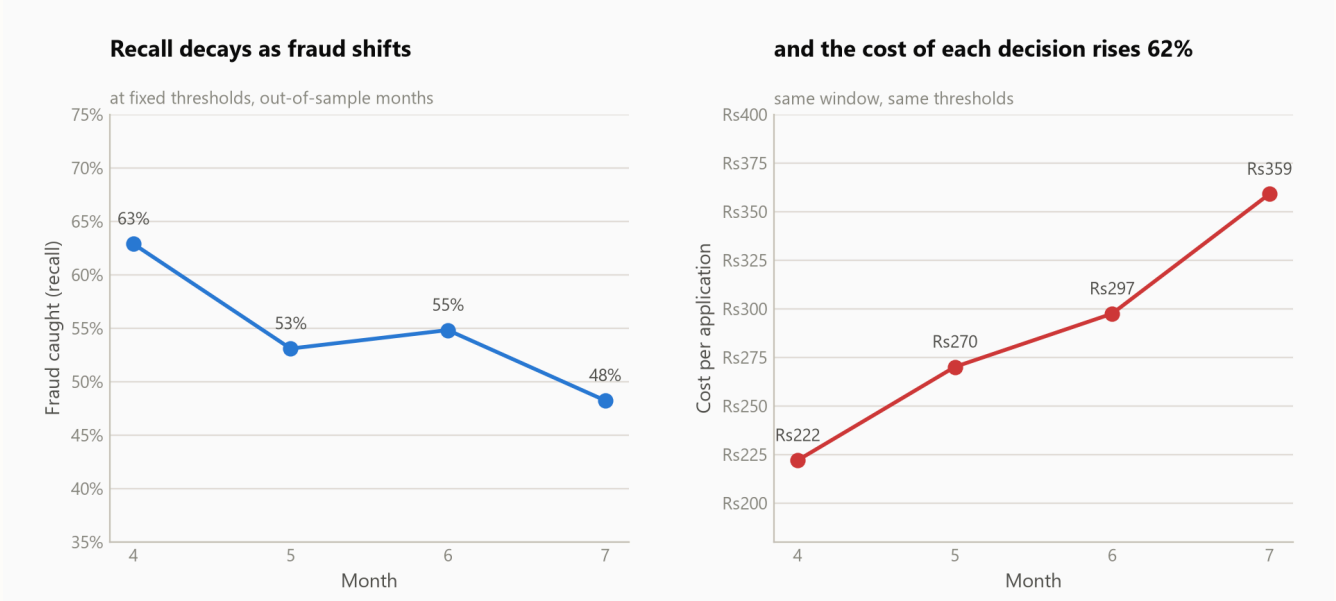
6 Finding IV — what decays is the operating point, not the model

Recall falls from 62.9% in month 4 to 48.2% in month 7. The instinctive reading is that the model is going stale and needs retraining. The data says otherwise.

MONTH	FRAUD RATE	ROC-AUC	RECALL	COST / APPLICATION
4	1.14%	0.898	62.9%	₹222
5	1.18%	0.886	53.1%	₹270
6	1.34%	0.885	54.8%	₹297
7	1.47%	0.879	48.2%	₹359

Out-of-sample months only. Months 0–3 were trained on; including them would present overfitting as though it were drift.

ROC-AUC moves by roughly 2.1% across this window. The model's ability to *rank* applications is essentially intact. What changed is that fraud prevalence rose from 1.14% to 1.47% while the thresholds stayed where they were set — so the same boundary now sits in a different place relative to the population.



Two measures, two charts, one shared x-axis — never a dual y-axis, which lets whoever draws it imply any correlation they like.

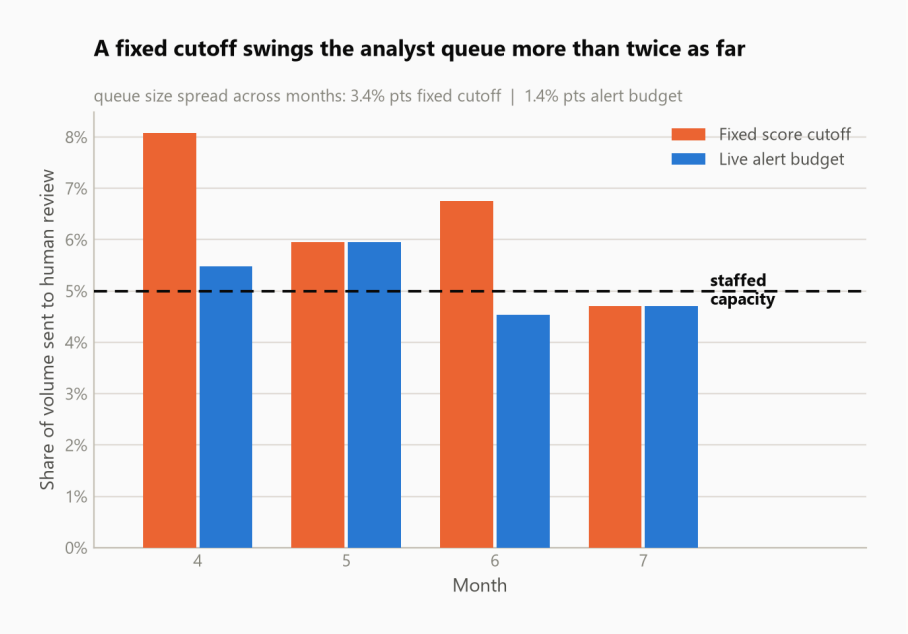
> WHY THE DISTINCTION IS WORTH MONEY
>
> "Retrain the model" is weeks of work and a model-risk sign-off. "Re-derive the thresholds" is a configuration change that takes effect on the next request. Diagnosing this incorrectly means paying the first price for a problem that only required the second.

7 The review queue, and the limitation the cost model hides

Section 2 traded 8,946 wrongly-blocked customers for 11,869 manual reviews. That trade assumes an analyst is available for each one. Here is what actually happens to the queue.

MONTH	FRAUD RATE	FIXED CUT-OFF	ALERT BUDGET	RECALL (FIXED / BUDGET)
4	1.14%	8.1%	5.5%	62.9% / 55.5%
5	1.18%	6.0%	6.0%	53.1% / 53.1%
6	1.34%	6.8%	4.5%	54.8% / 47.4%
7	1.47%	4.7%	4.7%	48.2% / 48.2%

Review rate against a team staffed for 5%. Red marks a month where the fixed cut-off overruns the staffing it was given.



An alert budget holds the queue near its target in most months — but not all of them. Month 5 overruns, and the chart says so.

THE WEAKEST POINT IN THIS DESIGN

The cost model charges ₹200 per review whether or not anyone performs it. So when the fixed cut-off overruns its staffing, it appears to win on recall — while spending analyst-hours that do not exist. A queue that overflows does not degrade gracefully; it degrades into applications sitting unreviewed until they time out. Pricing capacity overflow is the most valuable single change this system could receive.

8 Adaptation — including the result that did not work

Given the decay in Section 6, the natural response is to adapt. Four strategies were tested: leave it alone, recalibrate on the newest month, retrain the booster entirely, and hold a fixed alert budget without using any labels. Each adapts on month 6 and is evaluated on month 7.

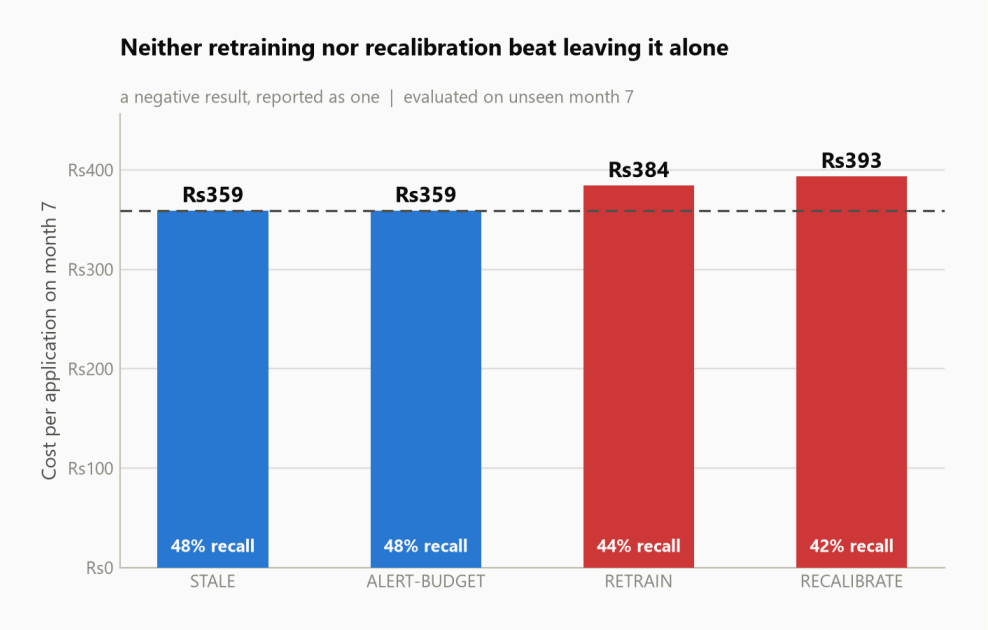
STRATEGY	RECALL	COST / APP	VS. STALE	ADAPT
STALE deployed as-is: trained months 0-3, calibrated on month 5	48.2%	₹359	–	0.0s
RECALIBRATE same booster; calibration and thresholds refitted on month 6	42.3%	₹393	+34	1.7s
RETRAIN booster refitted on months 0-6, then recalibrated	43.8%	₹384	+25	857.7s
ALERT-BUDGET stale model; review threshold set as a quantile of live scores (no labels)	48.2%	₹359	–	6.5s

Adapting on month 6, evaluated on month 7.

NEGATIVE RESULT, REPORTED AS ONE

No adaptation strategy beat doing nothing. Retraining the booster on an extra three months of data cost 14 minutes of compute and made the outcome *worse* — recall fell from 48.2% to 43.8%, and cost per application rose by ₹25.

The reason is visible in Section 6: discrimination was never the problem, so improving discrimination could not be the fix. Refitting on a period with a different fraud mix moved the thresholds to fit a month that had already passed. This is reported because a report that only contains experiments that worked is not evidence of judgement.



Four strategies, one evaluation month. The bar that does nothing is the one that wins.

9 Adaptation that does work — one confirmed case becomes a detector

Retraining failed because it is the wrong instrument for a fast-moving pattern. A supervised model only recognises fraud resembling its training labels; when a new pattern appears it is blind until enough cases are confirmed, labelled, retrained and redeployed — weeks during which the pattern runs unchecked.

The similarity layer closes that gap without touching the model. Each application is represented by **the leaves it occupies in the gradient-boosted ensemble**, and two applications are similar in proportion to how many trees route them to the same leaf. This uses the model's own learned notion of resemblance — including feature interactions — rather than a distance metric imposed on top of it. It is also directly interpretable, because the score has a plain reading: a similarity of 0.82 means *these two applications are routed to the same leaf by 82% of the trees in the model*.

The threshold was measured, not chosen

SIMILARITY CUT-OFF	FIRES ON GENUINE	FIRES ON FRAUD	LIFT
0.45	32.3%	75.5%	2.3×
0.55	5.3%	30.6%	5.8×
0.60	**1.3%**	**10.2%**	**7.9×**
0.65	0.3%	2.0%	6.2×

Measured on 4,000 held-out applications against a 400-case index. 0.60 peaks the lift. A lower cut-off fires on a third of all traffic at barely 2× lift — alert fatigue, not signal.

The escalation rule is deliberately narrow

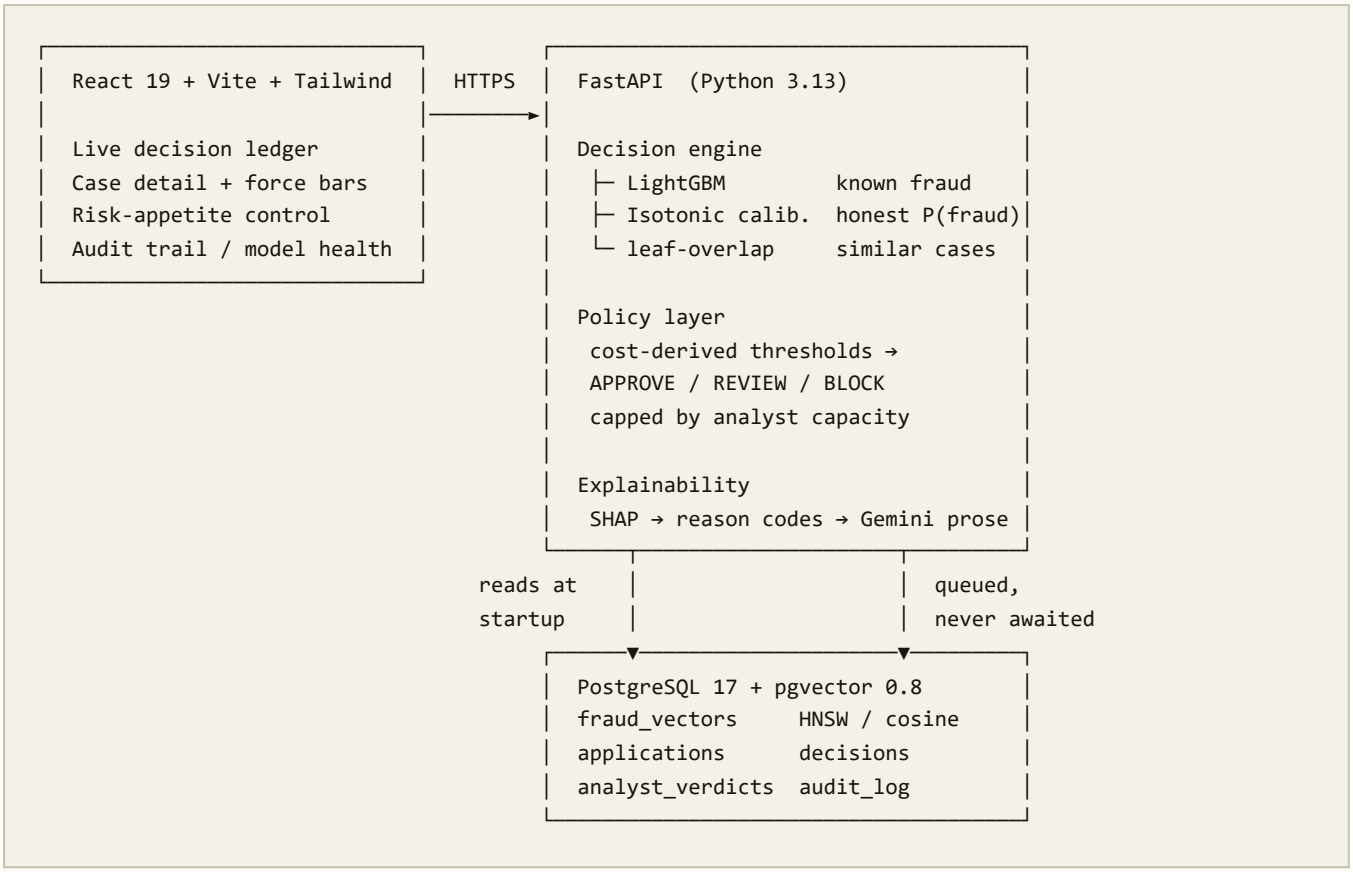
A match alone does not decide anything. The rule only ever *raises* a decision, never lowers one; it moves APPROVE to REVIEW and never straight to BLOCK; and it fires only on a match strength measured to carry 7.9× lift. At 8.9% precision, auto-blocking on similarity would decline roughly ten genuine applicants for every fraud stopped. The model's own decision is preserved in the response so an auditor can always see what was decided and what changed it.

```
BEFORE   decision=APPROVE   model_decision=APPROVE   escalated=False
         p(fraud)=0.0259  - a fraudulent application the model lets through
         an analyst confirms the fraud  ->  index 400 -> 401
AFTER    decision=REVIEW    model_decision=APPROVE   escalated=True
         reason: 100% match to confirmed fraud case APP-F482729F9C
                 (shares device_os, employment_status, housing_status)
         p(fraud)=0.0259  - unchanged, because the model was never retrained
```

The confirmed case is written to PostgreSQL before it enters the in-process index, so it survives a restart. Verified by killing the process:

```
before restart   fraud_vectors 401   index 401   decisions 562
   process killed and restarted
after  restart   fraud_vectors 401   index 401   decisions 562
                 index rebuilt from Postgres — the analyst's work survived
```

10 Architecture



What is deliberately kept off the decision path

Two things in this system are slow, and neither is allowed to make a decision slow.

The language model. Gemini turns SHAP attributions into analyst prose. It costs roughly 3.4 seconds — two orders of magnitude more than everything else combined — so it is never called during scoring. The console requests it only when an analyst opens a case, and a deterministic template always exists as a fallback. Replaying an application against the same model version yields an identical decision whether or not the provider was reachable. A decision that can only be explained when a third-party API is up is not auditable.

The audit write. A managed PostgreSQL is ~250ms away over the public internet — several decisions' worth of latency for one round trip. Writes are queued and drained by a background worker; the request thread pays `0.115ms`, measured. The cost of that choice is stated honestly in Section 12.

MEASURED	VALUE	CONDITIONS
Decision latency, p50	`46 ms`	paced as the console issues requests
Decision latency, p95	`86 ms`	100 requests, 402-case index, persistence on
Audit write, request thread	`0.115 ms`	serialise and enqueue
Narration, when requested	`~3.4 s`	off the decision path by design

Measured on a developer laptop with model, calibrator, SHAP and policy in one process. Sensitive to machine load: the same benchmark returned p50 118ms while unrelated build watchers consumed half the CPU.

11 Explainability, and the regulatory obligation behind it

A lender that declines an application generally has to say why. That is not a product preference; under adverse-action requirements it is a legal obligation, and it rules out any explanation method that cannot attribute a *specific decision* to *specific inputs*.

Aegis uses SHAP TreeExplainer, whose attributions are **additive**: the contributions sum to the model's output for that application. The force bars in the console are therefore the explanation itself, not an illustration of one. They are ordered by magnitude and rendered into adverse-action reason codes in that order — the ranking a regulator expects.

THE BOUNDARY THAT MAKES THIS DEFENSIBLE

The language model sits strictly *downstream* of the decision. The booster, the calibrator and the policy produce an outcome; SHAP explains that outcome; Gemini narrates the explanation. Nothing downstream of the policy can change what was decided. This ordering is enforced by the code structure and asserted by the smoke tests, not left as an intention.

Responsible AI

CONCERN	TREATMENT
Protected attributes	Age and employment status are present in the data and analysed for disparate impact rather than silently dropped — dropping a variable does not remove its proxies.
Explanation integrity	Additive SHAP; the LLM cannot alter a score, a threshold or an outcome.
Auditability	Every decision written to an append-only log; PostgreSQL rejects UPDATE and DELETE via trigger.
Human in the loop	The review band exists precisely so borderline cases reach a person rather than an automatic decline.
Honest reporting	Accuracy deliberately excluded; the demo stream discloses its own over-sampling and shows misses as well as catches.

> STATED PLAINLY
>
> Housing status carries 26.9% of total model gain. It is a legitimate stability signal and also a plausible proxy for socio-economic status. A production deployment would need a formal disparate-impact review before this model decided anything about a real person.

12 Limitations

Stated here rather than left to be discovered.

LIMITATION	CONSEQUENCE, AND WHAT WOULD FIX IT
Capacity overflow is not priced	The cost model charges ₹200 per review whether or not an analyst exists. This is why a fixed cut-off appears to beat the alert budget on recall — it spends hours it does not have. The most valuable single change available.
The dataset is synthetic	BAF is a CTGAN rendering of a real anonymised dataset under differential privacy. It preserves realistic structure, feature semantics and temporal drift, but it is not live production data.
Account opening ≠ lending origination	The fields are unambiguously credit-product ones — proposed credit limit, payment plan type, other cards held — so the domain match is close. There is no post-origination behaviour, so first-party bust-out fraud is out of scope.
No adaptation strategy beat inaction	Reported as a negative result in Section 8 rather than omitted.
Calibration produces tied probabilities	Isotonic regression yields heavily tied outputs, so quantile- and capacity-based thresholds are coarse; neither mechanism lands exactly on 5%. Exact targeting would need tie-breaking on the raw score.
The audit write is asynchronous	A process killed between enqueue and flush loses at most one half-second batch. Correct against a database across the internet; wrong against a co-located one.
No cloud deployment	Runs locally against managed PostgreSQL. Containerisation, IAM and a hosted runtime are designed but not built.
The analyst catch rate is assumed	90% is a configured assumption, not a measurement. It is configuration precisely so it can be replaced with an observed figure.
The browser holds the API key	Acceptable for a single-operator console, not for production. The production path is a session-authenticated backend-for-frontend holding the key server-side.

Engineering

PRACTICE	STATE
Unit tests	`86 passing`
End-to-end smoke checks	`24`
Live-database verification	`20 checks`
Secrets	environment only; none in source or version control
API contract	OpenAPI, generated from the code that serves it
Authentication	API key on every route, including metrics